

Failure (bad key + cost comparison) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume `client = OpenAI()` with `.env` already loaded.

Basic Version

Approach 1 — the direct way

```
# auth_and_cost_practice.py
import openai
from openai import OpenAI

client = OpenAI()

# part 1: bad key
bad_client = OpenAI(api_key="sk-fake-key-123")
try:
    bad_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": "hi"}],
    )
except openai.AuthenticationError:
    print("Login failed – check your API key.")

# part 2: cost comparison
short = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "Be helpful."},
        {"role": "user", "content": "What's the capital of France?"},
    ],
)

long_prompt = "You are a friendly, detailed assistant. " * 20
long = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": long_prompt},
```

```
        {"role": "user", "content": "What's the capital of France?"},  
    ],  
)  
  
print("Short prompt tokens:", short.usage.total_tokens)  
print("Long prompt tokens:", long.usage.total_tokens)
```

Expected output:

```
Login failed – check your API key.  
Short prompt tokens: 24  
Long prompt tokens: 168
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the same comparison, in named functions

```
# auth_and_cost_practice.py
import openai
from openai import OpenAI

client = OpenAI()

def try_bad_key() -> None:
    bad_client = OpenAI(api_key="sk-fake-key-123")
    try:
        bad_client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": "hi"}],
        )
    except openai.AuthenticationError:
        print("Login failed – check your API key.")

def compare_prompt_cost(question: str) -> None:
    short_prompt = "Be helpful."
    long_prompt = "You are a friendly, detailed, thorough assistant who explains

    short = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system", "content": short_prompt}, {"role": "user", "c
    )
    long = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system", "content": long_prompt}, {"role": "user", "c
    )

    print(f"Short system prompt: {short.usage.total_tokens} tokens")
    print(f"Long system prompt: {long.usage.total_tokens} tokens")
```

```
try_bad_key()  
compare_prompt_cost("What's the capital of France?")
```

Expected output:

```
Login failed – check your API key.  
Short system prompt: 24 tokens  
Long system prompt: 187 tokens
```

Difference from Basic: splitting into `try_bad_key()` and `compare_prompt_cost()` makes each concern independently reusable and testable, and full type hints document what `compare_prompt_cost` needs (a question string) and that it doesn't return anything — it just prints.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a computed dollar cost per call

```
# auth_and_cost_practice.py
import openai
from openai import OpenAI

client = OpenAI()

# gpt-4o-mini pricing (per 1M tokens) – check current pricing before using this f
PRICE_PER_1M_INPUT = 0.15
PRICE_PER_1M_OUTPUT = 0.60

def estimate_cost(usage) -> float:
    input_cost = (usage.prompt_tokens / 1_000_000) * PRICE_PER_1M_INPUT
    output_cost = (usage.completion_tokens / 1_000_000) * PRICE_PER_1M_OUTPUT
    return input_cost + output_cost

def compare_prompt_cost(question: str) -> None:
    short_prompt = "Be helpful."
    long_prompt = "You are a friendly, detailed, thorough assistant who explains

    short = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system", "content": short_prompt}, {"role": "user", "c
    )
    long = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system", "content": long_prompt}, {"role": "user", "c
    )

    print(f"Short system prompt: {short.usage.total_tokens} tokens, ~${estimate_c
    print(f"Long system prompt: {long.usage.total_tokens} tokens, ~${estimate_co

compare_prompt_cost("What's the capital of France?")
```

Expected output:

```
Short system prompt: 24 tokens, ~$0.000005
Long system prompt: 187 tokens, ~$0.000032
```

Approach 2 — a `BudgetTracker` that enforces a spending cap across many calls

```
# auth_and_cost_practice.py
class BudgetExceededError(Exception):
    """Raised when a call would push cumulative spend over the configured limit."""
    pass

PRICE_PER_1M_INPUT = 0.15
PRICE_PER_1M_OUTPUT = 0.60

def estimate_cost(usage) -> float:
    input_cost = (usage.prompt_tokens / 1_000_000) * PRICE_PER_1M_INPUT
    output_cost = (usage.completion_tokens / 1_000_000) * PRICE_PER_1M_OUTPUT
    return input_cost + output_cost

class BudgetTracker:
    def __init__(self, limit_dollars: float) -> None:
        self.limit_dollars = limit_dollars
        self.spent = 0.0

    def record(self, usage) -> float:
        cost = estimate_cost(usage)
        if self.spent + cost > self.limit_dollars:
            raise BudgetExceededError(
                f"This call would cost ~${cost:.6f}, bringing total spend to "
                f"~${self.spent + cost:.6f}, over the ${self.limit_dollars:.6f} limit"
            )
        self.spent += cost
        return cost

tracker = BudgetTracker(limit_dollars=0.00003)

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Say hello in one word."}],
)
```

```
print("Call 1 cost:", tracker.record(response.usage))

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Explain quantum computing in detail."}]
)
try:
    print("Call 2 cost:", tracker.record(response.usage))
except BudgetExceededError as e:
    print("Budget stopped a call:", e)
```

Expected output (exact costs vary by reply length):

```
Call 1 cost: 5.4e-06
Budget stopped a call: This call would cost ~$0.000041, bringing total spend to ~
```

Notice the second call already *happened* by the time `record()` raises — this tracks spend after the fact and stops the *next* call, not the one that just ran. A stricter version would estimate cost from the request before sending, but that requires counting input tokens with `tiktoken` (as in the Edge cases exercise) since you don't know the completion's length until after the call finishes.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate compares 2 calls' token counts side by side and stops there. Approach 1 turns those same counts into an actual dollar estimate — meaningful because input and output tokens are priced differently, so a bare token count can be misleading about actual cost. Approach 2 builds on Approach 1's `estimate_cost()` but answers the bigger design question: instead of reporting cost after one call, it accumulates cost *across* calls and refuses once a limit would be crossed — the difference between "here's what that cost" and "this literally cannot happen again until you raise the limit."

Which one should you actually ship? Approach 1's per-call cost estimate is worth logging in any real app — it's cheap, and it's the number you'd actually put in a report. Reach for Approach 2's `BudgetTracker` once a feature runs unattended or is exposed to many users, where "someone left a loop running overnight" or "one user hammered a feature" is a real, foreseeable risk, not a hypothetical one — exactly the situation this document's Build Task's chatbot will eventually be used in.